

Design Patterns

Eine schematische Lösung für ein häufig wiederkehrendes Problem

schematisch:

- kein Code
- kein Algorithmus
- sondern Skizzierung einer Lösung durch beteiligte Objekte und ihre Beziehungen

"Trickkiste" des erfahrenen Programmierers

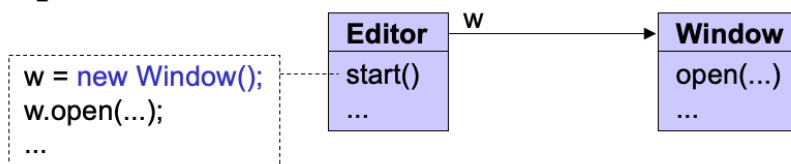
Bewährtes Architekturwissen

Erzeugende Muster

Factory Pattern

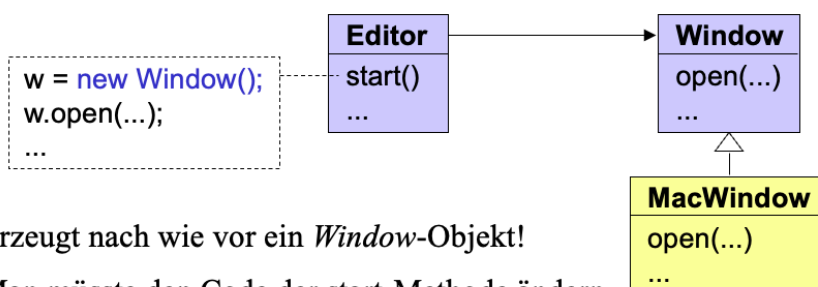
Das Factory Pattern ist ein Entwurfsmuster, das verwendet wird, um die Erstellung von Objekten zu vereinfachen, indem es eine abstrakte Fabrikklasse oder eine Schnittstelle bereitstellt, die von konkreten Fabrikklassen implementiert wird. Diese konkreten Fabrikklassen sind dafür verantwortlich, die konkreten Objekte zu erstellen. Auf diese Weise können Änderungen an der Art und Weise, wie Objekte erstellt werden, in einer einzigen Stelle vorgenommen werden, anstatt die Erstellung in jeder Klasse, die das Objekt verwendet, separat zu ändern.

Beispiel



Problem

- *Editor* ist auf *Window* festgelegt
- Wie bringt man den Editor dazu, mit neuen Fensterarten (z.B. *MacWindow*) zu arbeiten?



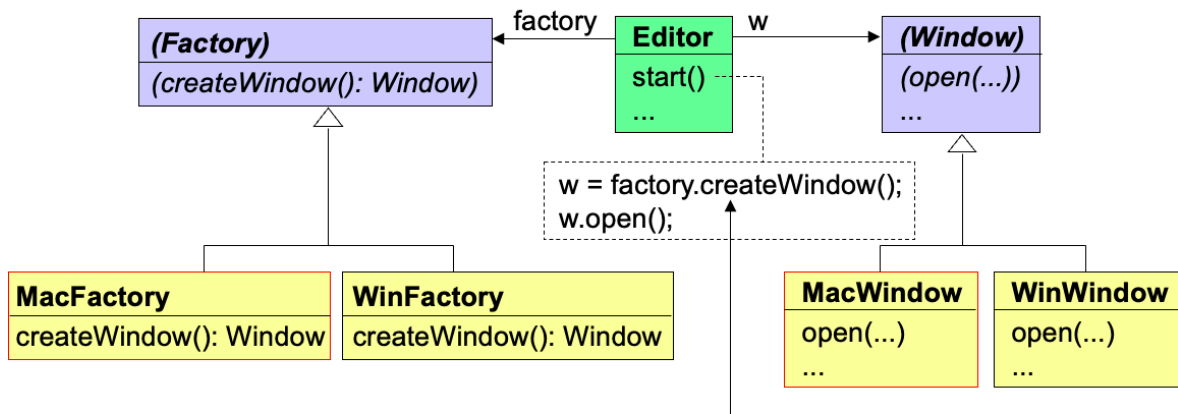
Erzeugt nach wie vor ein *Window*-Objekt!

Man müsste den Code der start-Methode ändern

- oft nicht möglich
- mühsam und fehleranfällig

Zweck Erzeugen eines Objekts mit bestimmter Schnittstelle
aber mit variablem dynamischem Typ

Lösung Objekt nicht mit *new* erzeugen, sondern von einem
"Fabrikobjekt" anfordern



Initialisierung am Programmbeginn
`factory = new MacFactory();`

lässt dynamischen Typ offen;
garantiert nur, dass irgendeine *Window*-Variante
geliefert wird

```
interface Shape {
    void draw();
}

class Rectangle implements Shape {
    @Override
    public void draw() {
        System.out.println("Inside Rectangle::draw() method.");
    }
}

class Square implements Shape {
    @Override
    public void draw() {
        System.out.println("Inside Square::draw() method.");
    }
}

class Circle implements Shape {
    @Override
    public void draw() {
        System.out.println("Inside Circle::draw() method.");
    }
}

class ShapeFactory {
    // use getShape method to get object of type shape
    public Shape getShape(String shapeType){
        if(shapeType == null){
            return null;
        }
    }
}
```

```

    }
    if(shapeType.equalsIgnoreCase("CIRCLE")){
        return new Circle();
    } else if(shapeType.equalsIgnoreCase("RECTANGLE")){
        return new Rectangle();
    } else if(shapeType.equalsIgnoreCase("SQUARE")){
        return new Square();
    }
    return null;
}
}

```

Prototype

Das Prototype Pattern ist ein Entwurfsmuster, das verwendet wird, um neue Objekte zu erstellen, indem es existierende Objekte kopiert. Es wird eine Vorlage verwendet, die als Prototyp bezeichnet wird, auf die das neue Objekt basiert.

Das Prototype Pattern besteht aus drei Teilen:

- Eine abstrakte Klasse oder Schnittstelle, die die Methode "clone" bereitstellt, die dazu verwendet wird, eine Kopie des Objekts zu erstellen.
- Eine konkrete Klasse, die von der abstrakten Klasse oder Schnittstelle erbt und die Methode "clone" implementiert, um eine tatsächliche Kopie des Objekts zu erstellen.
- Eine Klientenklasse, die das neue Objekt erstellt, indem es die Methode "clone" auf dem Prototypen aufruft.

```

package at.grimmer.prototype;

import java.util.Hashtable;

abstract class Shape implements Cloneable {

    private final String type;

    protected Shape(String type) {
        this.type = type;
    }

    abstract void draw();

    public String getType(){
        return type;
    }

    public Object clone() {
        Object clone = null;
        try {

```

```

        clone = super.clone();
    } catch (CloneNotSupportedException e) {
        e.printStackTrace();
    }
    return clone;
}

class Rectangle extends Shape {

    public Rectangle(){
        super("Rectangle");
    }

    @Override
    public void draw() {
        System.out.println("Inside Rectangle::draw() method.");
    }
}

class Square extends Shape {

    public Square(){
        super("Square");
    }

    @Override
    public void draw() {
        System.out.println("Inside Square::draw() method.");
    }
}

class Circle extends Shape {

    public Circle(){
        super("Circle");
    }

    @Override
    public void draw() {
        System.out.println("Inside Circle::draw() method.");
    }
}

class ShapeCache {

    private static Hashtable<String, Shape> shapeMap = new Hashtable<String, Shape>();

    public static Shape getShape(String shapeId) {
        Shape cachedShape = shapeMap.get(shapeId);
        return (Shape) cachedShape.clone();
    }
}

```

```

    public static void loadCache() {
        Circle circle = new Circle();
        shapeMap.put("CIRCLE", circle);

        Square square = new Square();
        shapeMap.put("SQUARE", square);

        Rectangle rectangle = new Rectangle();
        shapeMap.put("RECTANGLE", rectangle);
    }
}

public class PrototypeExample {

    public static void main(String[] args) {
        ShapeCache.loadCache();

        Shape clonedShape = (Shape) ShapeCache.getShape("CIRCLE");
        System.out.println("Shape : " + clonedShape.getType());

        Shape clonedShape2 = (Shape) ShapeCache.getShape("SQUARE");
        System.out.println("Shape : " + clonedShape2.getType());

        Shape clonedShape3 = (Shape) ShapeCache.getShape("RECTANGLE");
        System.out.println("Shape : " + clonedShape3.getType());
    }
}

```

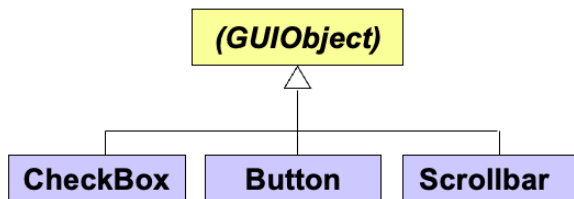
Strukturmuster

Familie

Zweck Zusammenfassung von Varianten

Lösung Abstrakte Klasse mit ihren Unterklassen

Beispiel



- Alle Familienmitglieder haben die gleiche Schnittstelle
- Jedes Familienmitglied kann als *GUIObject* verwendet werden

Adapter oder Wrapper

Zweck: Fremde Klasse zu einer bestehenden Familie kompatibel machen

Lösung: Neues Familienmitglied anlegen, das Familiennachrichten in Nachrichten an die fremde Klasse umsetzt (Forwarding)

Das Wrapper Design Pattern, auch bekannt als Adapter Pattern, ist ein Entwurfsmuster, das verwendet wird, um die Schnittstellen von bestehenden Klassen oder Objekten anzupassen, damit sie mit anderen Klassen oder Objekten zusammenarbeiten können. Es ermöglicht es, unterschiedliche Klassen und Objekte zusammenzuführen, die nicht miteinander kompatibel sind, indem es eine Schicht bereitstellt, die die Unterschiede ausgleicht.

Das Wrapper Design Pattern besteht aus drei Teilen:

- Eine abstrakte Klasse oder Schnittstelle, die die Methode bereitstellt, die angepasst werden muss.
- Eine konkrete Klasse, die von der abstrakten Klasse oder Schnittstelle erbt und die Methode implementiert, um die Anpassungen vorzunehmen.
- Eine Klientenklasse, die die Methode der konkreten Klasse aufruft, um die Anpassungen durchzuführen.

```
package at.grimmer.adapter;

interface MediaPlayer {
    void play(String audioType, String fileName);
}

class AdvancedMediaPlayer {
    public void playVlc(String fileName) {
        System.out.println("Playing vlc file. Name: " + fileName);
    }
}
```

```

    public void playMp4(String fileName) {
        System.out.println("Playing mp4 file. Name: " + fileName);
    }
}

class MediaAdapter implements MediaPlayer {

    private final AdvancedMediaPlayer advancedMusicPlayer;

    public MediaAdapter(AdvancedMediaPlayer advancedMusicPlayer){
        this.advancedMusicPlayer = advancedMusicPlayer;
    }

    @Override
    public void play(String audioType, String fileName) {

        if(audioType.equalsIgnoreCase("vlc")){
            advancedMusicPlayer.playVlc(fileName);
        }
        else if(audioType.equalsIgnoreCase("mp4")){
            advancedMusicPlayer.playMp4(fileName);
        }

    }
}

// VORGEGEBEN
class Stereo {
    private final MediaPlayer player;

    Stereo(MediaPlayer player) {
        this.player = player;
    }

    public void playAwesomeMusic() {
        player.play("mp4", "alle_meine_entchen.mp4");
    }
}

public class AdapterExample {
    public static void main(String[] args) {
        Stereo s = new Stereo(new MediaAdapter(new AdvancedMediaPlayer()));
        s.playAwesomeMusic();
    }
}

```

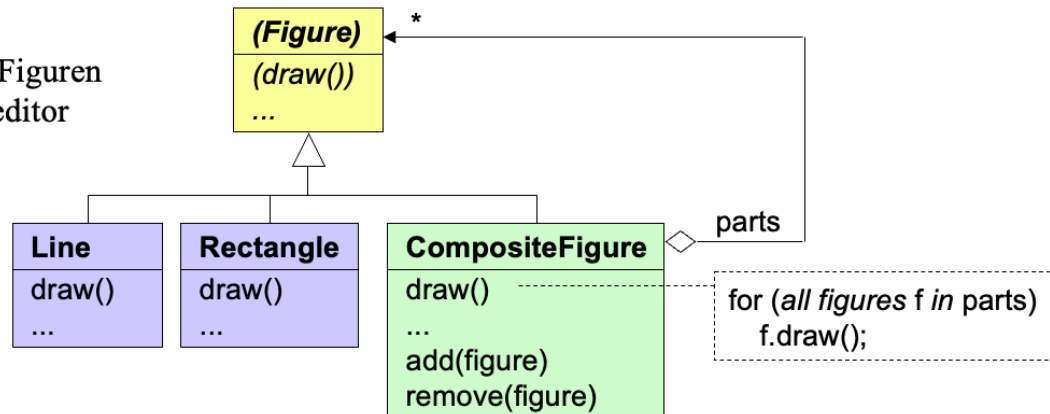
Composite Pattern

Zweck: Zusammengesetztes Objekt, das wie ein Einzelobjekt behandelt werden kann

Lösung: Zusammengesetztes Objekt ist Mitglied der Familie

Beispiel

Gruppieren von Figuren
in einem Grafikeditor



Alle *Figure*-Operationen sind auch auf Figurengruppen anwendbar

Oft ist auch ein *parent*-Zeiger von den Komponenten zum Kompositum nützlich

Das Composite Pattern ist ein Entwurfsmuster, das verwendet wird, um hierarchische Strukturen von Objekten zu erstellen, die aus einzelnen Objekten und Sammlungen von Objekten bestehen. Es ermöglicht es, sowohl einzelne Objekte als auch Sammlungen von Objekten auf die gleiche Weise zu behandeln, indem es eine abstrakte Basisklasse oder Schnittstelle bereitstellt, die von beiden implementiert wird.

```
package at.grimmer.composite;

import java.util.ArrayList;

abstract class Figure {
    public abstract void draw();
}

class Line extends Figure {
    public void draw() {
        System.out.println("Drawing a Line");
    }
}

class Rectangle extends Figure {
    public void draw() {
        System.out.println("Drawing a Rectangle");
    }
}

class CompositeFigure extends Figure {
    private ArrayList<Figure> figures = new ArrayList<Figure>();

    public void add(Figure f) {
```

```

        figures.add(f);
    }

    public void remove(Figure f) {
        figures.remove(f);
    }

    public void draw() {
        for (Figure f : figures) {
            f.draw();
        }
    }
}

public class CompositeExample {
    public static void main(String[] args) {
        CompositeFigure composite = new CompositeFigure();
        composite.add(new Line());
        composite.add(new Rectangle());

        composite.draw();
    }
}

```

Decorator Pattern

Zweck: Neues Verhalten zu einer Klasse hinzufügen, ohne die Klasse zu ändern und ohne eine Unterklasse zu bilden

Das Decorator-Muster in Java ist ein Entwurfsmuster, das es ermöglicht, Funktionalitäten von Klassen dynamisch und flexibel hinzuzufügen, indem es eine Abstraktion und deren Implementierungen durch Wrapper-Klassen ("Decorator") erweitert. Diese Wrapper-Klassen können die gleiche Schnittstelle wie die abstrakte Klasse implementieren und die ursprüngliche Implementierung durch Überschreiben der Methoden erweitern oder ergänzen.

```

package at.grimmer.decorator;

abstract class Kaffee {
    public abstract double getPreis();
}

class Espresso extends Kaffee {
    public double getPreis() {
        return 1.99;
    }
}

```

```

abstract class KaffeeDecorator extends Kaffee {
    protected Kaffee kaffee;
    public KaffeeDecorator(Kaffee kaffee) {
        this.kaffee = kaffee;
    }
    public abstract double getPreis();
}

class MilchDecorator extends KaffeeDecorator {
    public MilchDecorator(Kaffee kaffee) {
        super(kaffee);
    }
    public double getPreis() {
        return kaffee.getPreis() + 0.25;
    }
}

class ZuckerDecorator extends KaffeeDecorator {
    public ZuckerDecorator(Kaffee kaffee) {
        super(kaffee);
    }
    public double getPreis() {
        return kaffee.getPreis() + 0.15;
    }
}

public class DecoratorExample {
    public static void main(String[] args) {
        Kaffee meinKaffee = new ZuckerDecorator(new MilchDecorator(new Espresso()));
        System.out.println(meinKaffee.getPreis());
    }
}

```

Remember Filehandling in Java ;-)

Proxy Pattern

Das Proxy-Muster (auch "Proxy-Design-Muster" genannt) ist ein Entwurfsmuster in der objektorientierten Programmierung, das es ermöglicht, den Zugriff auf ein bestimmtes Objekt zu steuern oder zu überwachen, indem man ein Proxy-Objekt einsetzt, das die Anfragen an das eigentliche Objekt weiterleitet.

Sehr ähnlich zum Decorator Pattern

```

package at.grimmer.proxy;

/**
 * Interface for images which can be displayed

```

```

*/
public interface Image {
    /**
     * Displays the image.
     */
    public void displayImage();
}

package at.grimmer.proxy;

public class RealImage implements Image {
    private String filename;

    public RealImage(String filename) {
        this.filename = filename;
        loadImageFromDisk();
    }

    private void loadImageFromDisk() {
        // Potentially expensive operation
        System.out.println("Loading " + filename);
    }

    public void displayImage() {
        System.out.println("Displaying " + filename);
    }
}

package at.grimmer.proxy;

public class ProxyImage implements Image {
    private String filename;
    private Image image;

    public ProxyImage(String filename) {
        this.filename = filename;
    }

    public void displayImage() {
        if (image == null) {
            image = new RealImage(filename); // load only on demand
        }
        image.displayImage();
    }
}

package at.grimmer.proxy;

/**
 * Example which simulates lazy loading of images. Images are loaded from file when they are
 * displayed for the first time.
 */

```

```

class ProxyTest {
    @SuppressWarnings("unused")
    public static void main(String[] args) {
        Image image1 = new ProxyImage("HiRes_10MB_Photo1");
        Image image2 = new ProxyImage("HiRes_10MB_Photo2");
        Image image3 = new ProxyImage("HiRes_10MB_Photo3");

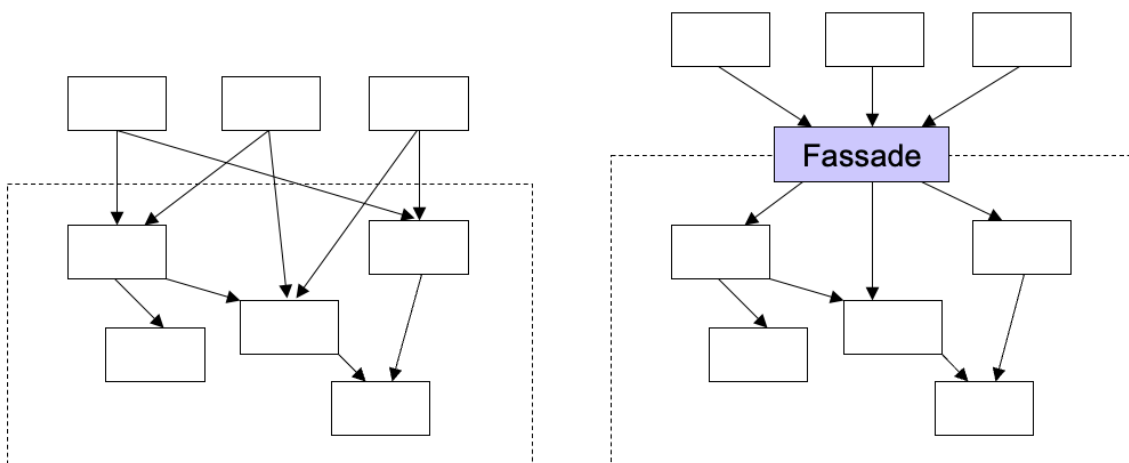
        image1.displayImage(); // loading necessary
        image2.displayImage(); // loading necessary
        image2.displayImage(); // no loading necessary; already done
        // image3 is never displayed so it is not loaded at all
    }
}

```

Fassade

Zweck Ansprechpartner für ein Subsystem sein

Lösung Künstliches Objekt als Vermittler zwischen Benutzern und Subsystem



Beispiel

- Schnittstelle zu einem Statistikpaket: Benutzer kann Statistikfunktionen über eine einheitliche Schnittstelle verwenden

Vorteile

- Benutzer müssen die Einzelteile des Systems nicht verstehen
- Fassade kann maßgeschneiderte Funktionen für verschiedene Benutzer anbieten
- Entkopplung: Subsystem kann sich ändern, ohne dass sich die Fassade ändert

Das Fassade-Muster (auch "Fassade-Design-Muster" genannt) ist ein Entwurfsmuster in der objektorientierten Programmierung, das es ermöglicht, die Komplexität von einer Gruppe von Unterklassen oder einem System zu verbergen, indem man eine einzige, vereinfachte Klasse als Schnittstelle bereitstellt.

```

package at.grimmer.fassade;

class Playlist {
    public void play() {

```

```

        // Code zum Starten der Wiedergabe
    }
    public void stop() {
        // Code zum Stoppen der Wiedergabe
    }
    public void nextTrack() {
        // Code zum Wechseln zum nächsten Track
    }
    // weitere Methoden
}

class Equalizer {
    public void setBass(int level) {
        // Code zum Einstellen des Basspegels
    }
    public void setTreble(int level) {
        // Code zum Einstellen des Höhenpegels
    }
    // weitere Methoden
}

class Lautstaerkeregler {
    public void setVolume(int level) {
        // Code zum Einstellen der Lautstärke
    }
    // weitere Methoden
}

class MusikspielerFassade {
    private Playlist playlist;
    private Equalizer equalizer;
    private Lautstaerkeregler volume;

    public MusikspielerFassade(Playlist playlist, Equalizer equalizer, Lautstaerkeregler
volume) {
        this.playlist = playlist;
        this.equalizer = equalizer;
        this.volume = volume;
    }

    public void play() {
        playlist.play();
    }
    public void stop() {
        playlist.stop();
    }

    public void setRockMusicMode() {
        equalizer.setBass(50);
        equalizer.setTreble(60);
        volume.setVolume(100);
    }
}

```

```

    public void setClassicMusicMode() {
        equalizer.setBass(20);
        equalizer.setTreble(50);
        volume.setVolume(100);
    }
}

public class FassadeExample {
    public static void main(String[] args) {

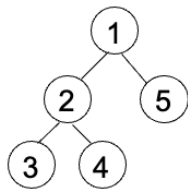
    }
}

```

Iterator Pattern

breits bekannt - siehe Unterlagen

Als Übung:



```

class Node {
    int value;
    Node left, right;
}

```

```

class Iterator {
    Stack stack = new Stack();
    Iterator(Node root) {
        if (root != null) stack.push(root);
    }
    int next() {
        Node p = stack.pop();
        if (p.right != null) stack.push(p.right);
        if (p.left != null) stack.push(p.left);
        return p.value;
    }
    boolean hasNext() {
        return !stack.empty();
    }
}

```

Simulation

	<i>Stack</i>	
new Iterator(root);	1	
iter.next();	5 2	⇒ 1
iter.next();	5 4 3	⇒ 2
iter.next();	5 4	⇒ 3
iter.next();	5	⇒ 4
iter.next();		⇒ 5

Visitor Pattern

Das Visitor-Muster ist ein Entwurfsmuster in der objektorientierten Programmierung, das es ermöglicht, neues Verhalten auf eine Gruppe von Objekten anzuwenden, ohne deren Klassen zu ändern. Es erreicht dies, indem es eine separate Klasse einführt, die das Verhalten implementiert, die sogenannte "Visitor"-Klasse.

In Java kann das Visitor-Muster durch die Verwendung von generischen Typparametern und Interfaces implementiert werden.

```
package at.grimmer.visitor;

interface Form {
    public void accept(FormVisitor visitor);
}

class Kreis implements Form {
    private double radius;
    public Kreis(double radius) {
        this.radius = radius;
    }
    public double getRadius() {
        return radius;
    }
    public void accept(FormVisitor visitor) {
        visitor.visit(this);
    }
}

class Rechteck implements Form {
    private double breite;
    private double hoehe;
    public Rechteck(double breite, double hoehe) {
        this.breite = breite;
        this.hoehe = hoehe;
    }
    public double getBreite() {
        return breite;
    }
    public double getHoehe() {
        return hoehe;
    }
    public void accept(FormVisitor visitor) {
        visitor.visit(this);
    }
}

class Dreieck implements Form {
    private double seite1;
    private double seite2;
    private double seite3;
    public Dreieck(double seite1, double seite2, double seite3) {
        this.seite1 = seite1;
        this.seite2 = seite2;
        this.seite3 = seite3;
    }
    public double getSeite1() {
```

```

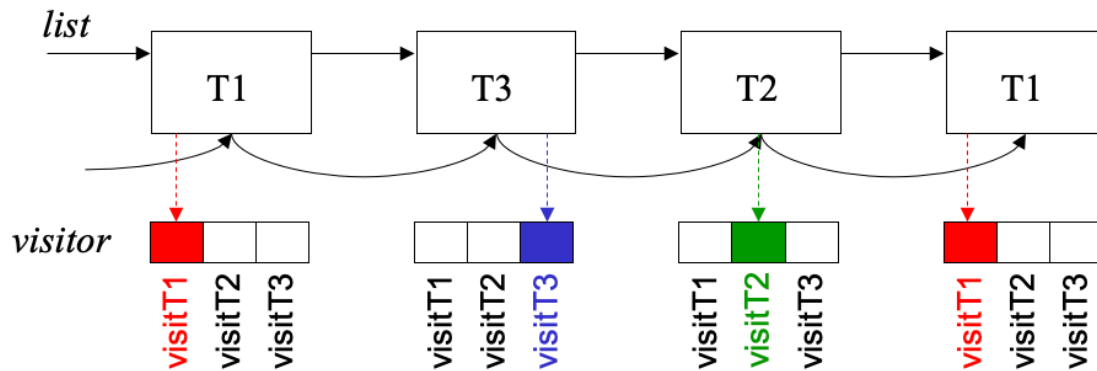
        return seite1;
    }
    public double getSeite2() {
        return seite2;
    }
    public double getSeite3() {
        return seite3;
    }
    public void accept(FormVisitor visitor) {
        visitor.visit(this);
    }
}

interface FormVisitor {
    public void visit(Kreis kreis);
    public void visit(Rechteck rechteck);
    public void visit(Dreieck dreieck);
}

class BerechneFlächeVisitor implements FormVisitor {
    public void visit(Kreis kreis) {
        double fläche = Math.PI * kreis.getRadius() * kreis.getRadius();
        System.out.println("Fläche des Kreises: " + fläche);
    }
    public void visit(Rechteck rechteck) {
        double fläche = rechteck.getBreite() * rechteck.getHoehe();
        System.out.println("Fläche des Rechtecks: " + fläche);
    }
    public void visit(Dreieck dreieck) {
        double s = (dreieck.getSeite1() + dreieck.getSeite2() + dreieck.getSeite3()) / 2;
        double fläche = Math.sqrt(s * (s - dreieck.getSeite1()) * (s - dreieck.getSeite2())
* (s - dreieck.getSeite3()));
        System.out.println("Fläche des Dreiecks: " + fläche);
    }
}

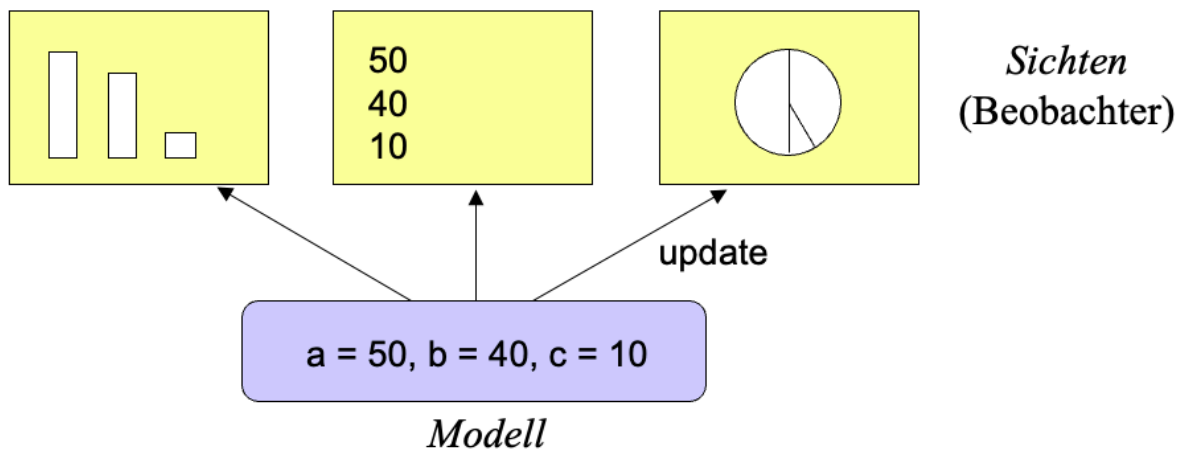
public class SimpleVisitorExample {
    public static void main(String[] args) {
        Form[] formen = new Form[]{new Kreis(5), new Rechteck(3, 4), new Dreieck(3, 4, 5)};
        FormVisitor visitor = new BerechneFlächeVisitor();
        for (Form form : formen) {
            form.accept(visitor);
        }
    }
}

```



Model View Controller oder Beobachter/Obverster Pattern

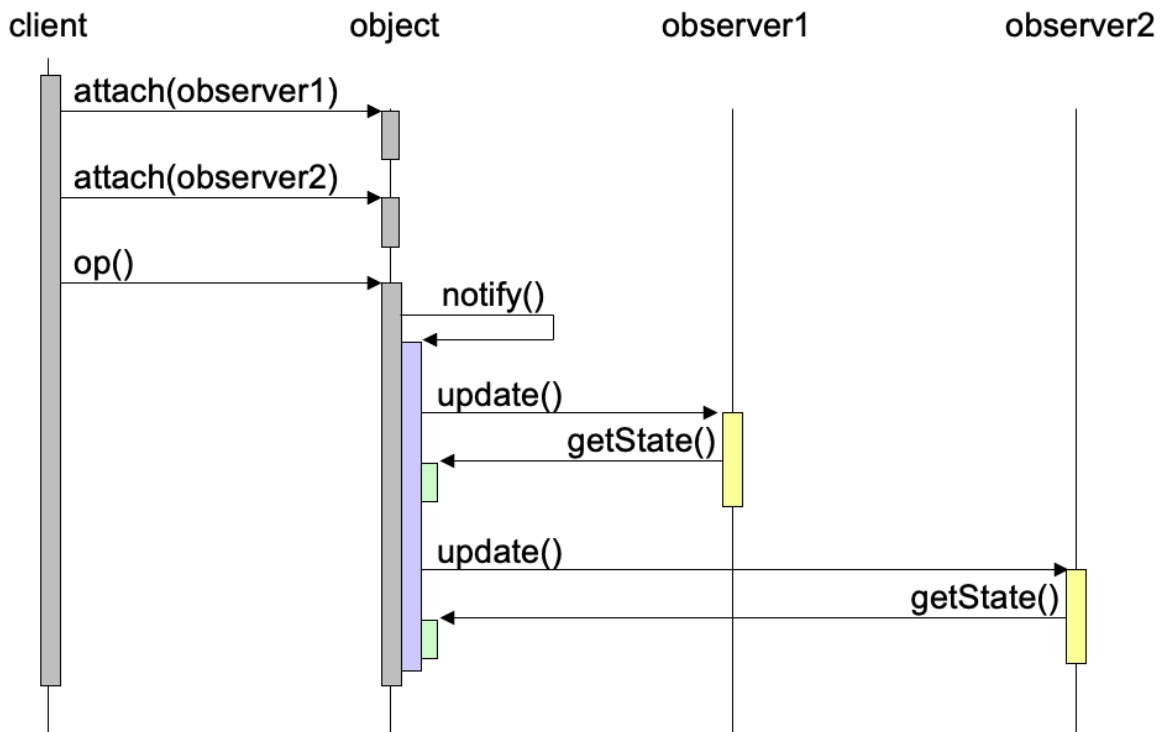
Zweck: Objekt, das benachrichtigt wird, wenn sich ein anderes Objekt ändert



Sobald das Modell geändert wird, werden alle Sichten benachrichtigt

Zweck: alle Sichten sind immer konsistent

- Sichten melden sich beim Modell als Beobachter an
- Modell benachrichtigt zu gegebener Zeit alle seine Beobachter



Code Beispiel zu lange: siehe Points-Example in IntelliJ

Chain of Responsibilities

Das Chain of Responsibilities (Kette der Verantwortung) Muster ist ein Entwurfsmuster in der objektorientierten Programmierung, das es ermöglicht, die Verantwortung für die Verarbeitung einer Anfrage zwischen einer Kette von Objekten aufzuteilen. Es ermöglicht es, dass jedes Objekt in der Kette entscheiden kann, ob es die Anfrage verarbeiten kann oder ob sie an das nächste Objekt in der Kette weitergegeben werden sollte.

```

package at.grimmer.chainofresponsibilities.simple;

class Anfrage {
    private final Anfragetyp anfragetyp;
    private final String beschreibung;

    Anfrage(Anfragetyp anfragetyp, String beschreibung) {
        this.anfragetyp = anfragetyp;
        this.beschreibung = beschreibung;
    }

    public Anfragetyp getAnfragetyp() {
        return anfragetyp;
    }

    public String getBeschreibung() {

```

```

        return beschreibung;
    }
}

enum Anfragetyp {
    TECHNISCH, RÜCKERSTATTUNG, BESCHWERDE
}

abstract class AnfrageBearbeiter {
    protected AnfrageBearbeiter nächster;

    public void setNächster(AnfrageBearbeiter nächster) {
        this.nächster = nächster;
    }

    public void bearbeiteAnfrage(Anfrage anfrage) {
        if (kannAnfrageBearbeiten(anfrage)) {
            bearbeite(anfrage);
        } else if (nächster != null) {
            nächster.bearbeiteAnfrage(anfrage);
        }
    }

    protected abstract boolean kannAnfrageBearbeiten(Anfrage anfrage);
    protected abstract void bearbeite(Anfrage anfrage);
}

class TechnischerBearbeiter extends AnfrageBearbeiter {
    protected boolean kannAnfrageBearbeiten(Anfrage anfrage) {
        return anfrage.getAnfragetyp() == Anfragetyp.TECHNISCH;
    }

    protected void bearbeite(Anfrage anfrage) {
        System.out.println("Technischer Bearbeiter bearbeitet Anfrage: " +
            anfrage.getBeschreibung());
    }
}

class RückerstattungsBearbeiter extends AnfrageBearbeiter {
    protected boolean kannAnfrageBearbeiten(Anfrage anfrage) {
        return anfrage.getAnfragetyp() == Anfragetyp.RÜCKERSTATTUNG;
    }

    protected void bearbeite(Anfrage anfrage) {
        System.out.println("Rückerstattungs Bearbeiter bearbeitet Anfrage: " +
            anfrage.getBeschreibung());
    }
}

class BeschwerdeBearbeiter extends AnfrageBearbeiter {
    protected boolean kannAnfrageBearbeiten(Anfrage anfrage) {
        return anfrage.getAnfragetyp() == Anfragetyp.BESCHWERDE;
    }
}

```

```

        protected void bearbeite>Anfrage anfrage) {
            System.out.println("Beschwerde Bearbeiter bearbeitet Anfrage: " +
anfrage.getBeschreibung());
        }
    }

public class CORSSimpleExample {
    public static void main(String[] args) {
        AnfrageBearbeiter technischerBearbeiter = new TechnischerBearbeiter();
        AnfrageBearbeiter rückerstattungsBearbeiter = new RückerstattungsBearbeiter();
        AnfrageBearbeiter beschwerdeBearbeiter = new BeschwerdeBearbeiter();

        technischerBearbeiter.setNächster(rückerstattungsBearbeiter);
        rückerstattungsBearbeiter.setNächster(beschwerdeBearbeiter);

        Anfrage anfrage1 = new Anfrage>Anfragetyp.TECHNISCH, "Technisches Problem mit dem
WLAN");
        Anfrage anfrage2 = new Anfrage>Anfragetyp.RÜCKERSTATTUNG, "Rückerstattung des
Kaufpreises verlangt");
        Anfrage anfrage3 = new Anfrage>Anfragetyp.BESCHWERDE, "Beschwerde über den
Kundenservice");

        technischerBearbeiter.bearbeiteAnfrage(anfrage1);
        technischerBearbeiter.bearbeiteAnfrage(anfrage2);
        technischerBearbeiter.bearbeiteAnfrage(anfrage3);
    }
}

```